

The filter

B02 — inferring a charge nobody can measure, from a voltage and a current that both lie.

What a Kalman filter is

A weighted average between something you PREDICTED and something you MEASURED, weighted by how much you trust each. Everything else is bookkeeping for those weights.

You are walking a corridor with your eyes shut, counting paces. Counting is smooth and precise over a few steps, but the error grows without bound — you will never notice you are drifting left. Occasionally you open your eyes and glimpse a doorway through fog: blurry, but it does not drift.

counting paces	coulomb counting — smooth, drifts on bias
glimpsing the door	terminal voltage — noisy, but anchored

The six lines

ONE state, so there are no matrices — all of it is scalar arithmetic.

PREDICT

$$\hat{z}^- = z - \frac{\eta I \Delta t}{3600 Q}$$

where counting says you are

$$P^- = P + Q_{proc}$$

and how much less sure you are for guessing

CORRECT

$$H = \frac{dOCV}{dz}$$

the slope, at the PREDICTED state

$$K = \frac{P^- H}{H^2 P^- + R_{meas}}$$

how far to move toward the meter

$$z = \hat{z}^- + K(V_{meas} - \hat{V})$$

blend

$$P = (1 - KH) P^-$$

confidence recovers

2. The symbols, in code

Every quantity in the six lines, and what it is called in src/ekf.cpp.

Members already exist — ASSIGN them. Writing ``double soc_ = ...`` declares a new local that shadows the member, and the filter silently never learns.

MEMBERS — assign, never re-declare

Z `soc_`
 P `variance_`
 K `gain_`
 Q_{proc} `q_proc_`
 R_{meas} `r_meas_`

PARAMETERS

I `current_a`
 V_{meas} `terminal_voltage_v`
 Δt `dt_s`

CONSTANTS — from params::

η `COULOMBIC_EFFICIENCY`
 Q `CAPACITY_AH`
 R `R_INTERNAL_OHM`
 Z_{min} `SOC_MIN / SOC_MAX`

LOCALS — yours to create

H `h_v_per_soc`
 $\hat{V}$ `predicted_voltage_v`
 y `residual_v`

TWO different R's, and the equations write both the same way: `r_meas_` is a variance in volts-squared, `R_INTERNAL_OHM` is a resistance in ohms. Swap them and the filter still runs.

The shape, with the names in place

Straight-line code. No loops (one state means no matrices), no if-else (`std::clamp` absorbs it), no guard (the denominator cannot reach zero).

```
double Ekf::update(double current_a,
                   double terminal_voltage_v, double dt_s) {
    // predict
    soc_      = ...;
    variance_ = ...;

    // correct
    const double h_v_per_soc      = docv_dsoc(soc_);
    const double predicted_voltage_v = ...;
    const double residual_v
        = terminal_voltage_v - predicted_voltage_v;

    gain_      = ...;
    soc_      = ...;
    variance_ = ...;

    soc_ = std::clamp(soc_, params::SOC_MIN, params::SOC_MAX);
    return soc_;
}
```

ORDER IS MEANING. H must be taken AFTER the predict step — it is the slope at the PREDICTED state. Hoist it above and you have linearised about the old estimate: a different, worse filter. No compiler catches this.

Never clamp the variance — squeezing a confidence lies to the filter.

3. The gain, and what 'extended' means

K is the only interesting quantity. Everything else serves it.

$$K = \frac{PH}{H^2P + R_{meas}}$$

Near 0, the filter ignores the voltmeter. Near 1, it follows it completely.

Between, it weighs evidence — which is the entire behaviour worth having.

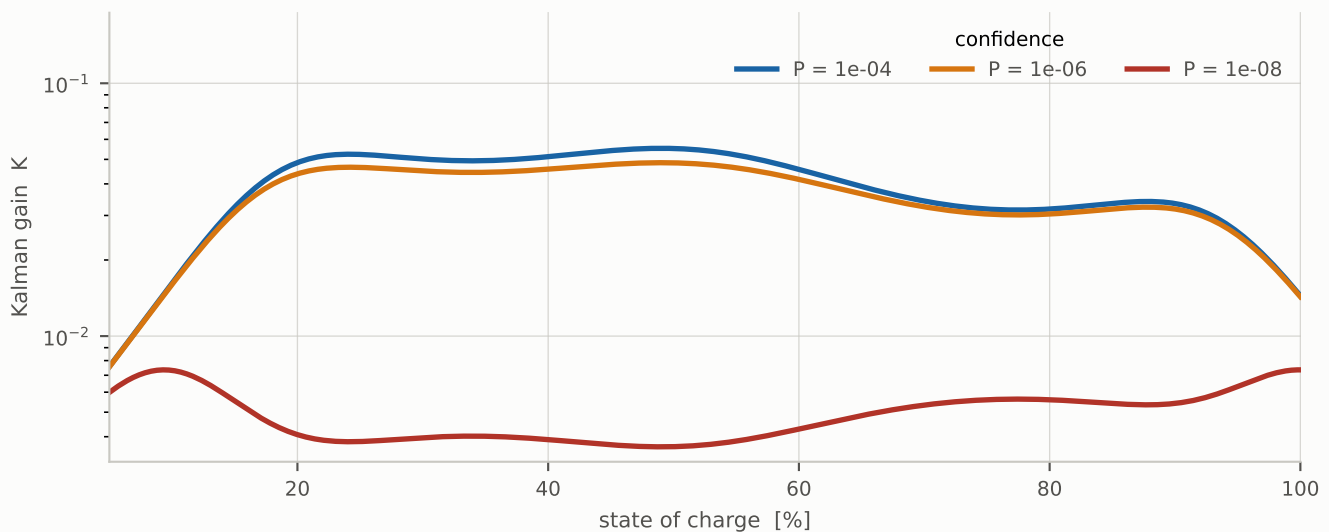
Why the chemistry outranks the instrument

H is the OCV slope. As $H \rightarrow 0$, the numerator vanishes faster than the denominator and the gain collapses:

$$H \rightarrow 0 \implies K \rightarrow 0$$

A flat curve means the filter learns NOTHING from a measurement, however good the voltmeter. That is not a tuning problem — it is the chemistry, and it is why LFP estimators are known to diverge.

Gain across the charge range. A less certain filter (larger P) leans harder on the meter.



What the E in EKF is doing

The measurement is a NONLINEAR function of the state — the OCV curve — so it is linearised at each step by taking its slope at the current estimate.

$$V = \text{OCV}(z) - IR \implies H = \left. \frac{\partial V}{\partial z} \right|_{\hat{z}}$$

H IS the extension. That is the whole difference from a plain Kalman filter, and the reason the slope has been the thread through all of this.

4. What to expect

Recorded in advance, so the result is not mistaken for a failure.

A positive bias over-reports the discharge, so COULOMB COUNTING runs below the truth. The FILTERED error settles slightly POSITIVE: the same bias enters V_{pred} through the IR term and pulls the other way.

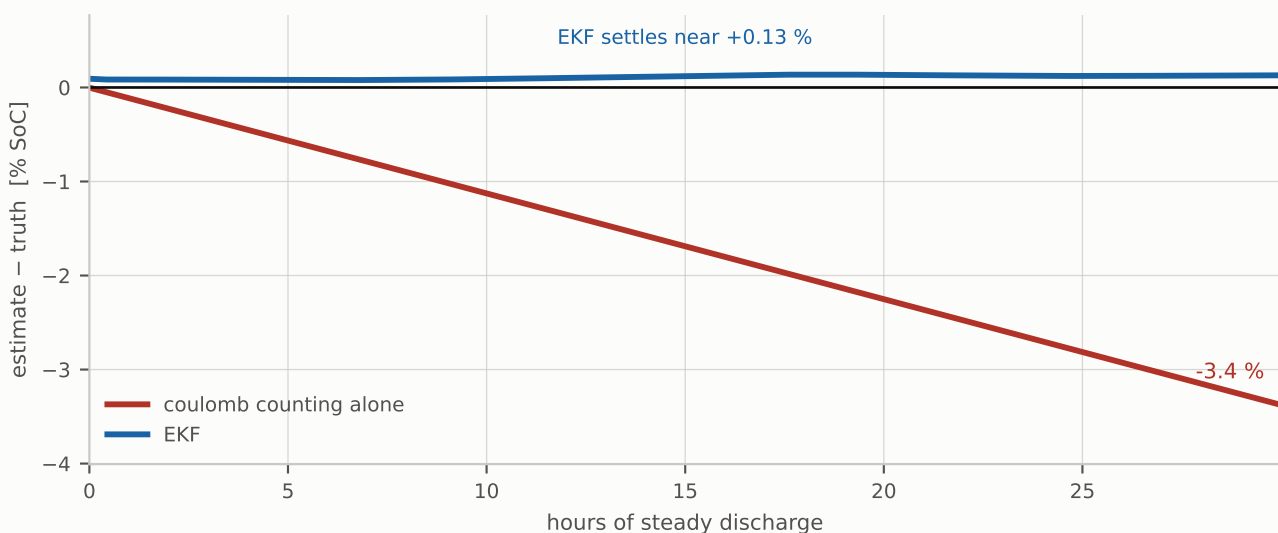
The bias is NOT in the state vector. This filter has one state, the charge, so it has nothing to estimate the bias WITH — and it will not remove it.

What it will do is reach equilibrium, where the voltage correction each step exactly cancels the drift the biased current injected. The error stops growing and stays put, at

$$e_{\infty} \approx \frac{b_I R}{H} - \frac{\eta b_I \Delta t}{3600 Q K H}$$

+0.00149 (IR) - 0.00022 (drift) = +0.127 % predicted, +0.129 % measured below.

A 2.0 A bias on a 1689 Ah pack, 40 A steady discharge.



UNBOUNDED drift becomes BOUNDED error. That is the finding — not that the estimate is perfect, but that it stops getting worse.

	after 30 h	over one 354 h night
coulomb counting	-3.4 %	-42 % and falling
EKF	+0.13 %	bounded — it stops here

If the residual is too large

The fix is NOT better sensors and NOT more tuning. It is to give the filter the bias to estimate — a second state, $[z, b_I]$, which makes it a 2x2 problem and drives the residual toward zero. Serious battery management does the same with R . Not first — but recognise the symptom.

Sources: Plett, G. L. (2004), 'Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs', Parts 1-3, J. Power Sources 134(2):262-276. Every number is computed from config.py. Regenerate with docs/make_ekf_formulas.py.